BACK MATTER

Appendix C: A Worked Organizational Specification

The Riverside Clinics booking feature, specified completely enough for a team that hasn't read this book to implement it correctly.

Chapter 2 gave the booking feature a vision. Chapter 3 turned that vision into requirements and a use case. Chapter 4 gave the use case a domain model. Chapter 5 assembled the four layers into one document. None of those chapters wrote the specification out in the form an implementing team actually receives it: every entity's fields, the exact sequence of a booking request, the rules a database has to enforce, the notifications a patient should expect, and the values a status field is allowed to hold. This appendix writes that version.

This appendix borrows a name from the internet standards Appendix B discusses and calls a document shaped like this one an internal RFC. The borrowing is this book's analogy, not a practice organizations already follow. This book has used the word specification throughout, and keeps using it here. The two names describe the same artifact: a precise, technology-independent statement of what a system must do, versioned and owned the way a codebase currently is, implementable by a Java team, a Go team, or an AI agent working from nothing else.

What follows: scope, the entities the feature depends on, the booking protocol step by step, the consistency rules a datastore must hold, notification behavior, the enumerations that constrain valid state, implementation notes for whoever builds it, and a proposed version 1.1 showing how the specification itself changes when the business asks for more.

Overview and Scope

> **Riverside Clinics Appointment Scheduling Specification**
>
> `Version: 1.0`
>
> `Status: Active`
>
> `Owners: Clinic Operations, Patient Services Engineering`
>
> `Last revised: 2026-01-12`

This specification defines the business protocol for scheduling a patient appointment at a Riverside Clinics location. Any system that lets a patient or a staff member create, cancel, or view an appointment must conform to it, whether that system is a public booking website, a phone-based intake tool, or a front-desk terminal. The specification doesn't mention programming language, deployment platform, or storage engine. Two conforming implementations may share no code at all and still interoperate, because agreement lives in the entities, the rules, and the messages passed between them.

IN SCOPE

- Patient-initiated appointment booking
- Staff-initiated appointment creation
- Appointment cancellation
- Doctor and clinic availability
- Concurrent-booking conflict resolution

OUT OF SCOPE

- Patient medical records
- Billing and insurance
- Telehealth session delivery
- Rescheduling as a separate flow (v1.0 handles a reschedule as a cancellation followed by a new booking)

Telehealth and rescheduling are visible gaps on purpose. Each is work this specification leaves for whoever writes the next version.

Entities: Patient, Doctor

Chapter 4 named Patient, Doctor, Clinic, Appointment, and TimeSlot as an excerpt of Riverside's domain model, sufficient to make one point about entities and value objects. A specification an implementing team can build from needs every field the protocol below actually touches. What follows extends that excerpt without contradicting it: each entity keeps the identity, attributes, and relationships Chapter 4 established, with the additional fields the protocol requires.

Patient (Entity)

- `patient_id`: unique identifier, immutable once assigned.
- `name`: string.
- `email`: string, must conform to the address syntax in RFC 5322.
- `phone`: string, ITU-T E.164 format.
- `date_of_birth`: date, must be in the past.
- `medical_record_number`: unique across all patients (Chapter 4).
- `registration_date`: timestamp, RFC 3339 format.

Relationship: has many Appointments.

Doctor (Entity)

- `doctor_id`: unique identifier.
- `name`: string.
- `specialty`: enumeration, Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry (Chapter 4).
- `years_licensed`: integer, non-negative.

Relationship: works at one or more Clinics, many-to-many (Chapter 4). A booking request must therefore name which clinic a doctor is being requested at; a doctor's record alone doesn't tell the system where they'll be that day.

Relationship: has many UnavailabilityWindows, the value object defined below.

Entities: Clinic, Appointment

Clinic (Entity)

- clinic_id: unique identifier.
- name: string.
- business_hours: the days and hours the clinic accepts appointments (Chapter 4). v1.0 fixes this to Monday through Friday, 08:00 to 17:00, clinic-local time.

Relationship: employs many Doctors.

Appointment (Entity)

- appointment_id: unique identifier.
- patient_id: reference to Patient.
- doctor_id: reference to Doctor.
- clinic_id: reference to Clinic. Must be one of the clinics the referenced Doctor works at.
- scheduled_slot: TimeSlot value object (Chapter 4).
- duration_minutes: 15, 30, 45, or 60.
- status: enumeration, defined in full under Enumerations.
- reason_for_visit: string, must not be empty.
- confirmation_number: an opaque identifier returned to whoever booked the appointment.
- created_timestamp: RFC 3339 timestamp.
- created_by_actor: enumeration, Patient or Staff.

Relationship: involves exactly one Patient, exactly one Doctor, at exactly one Clinic.

Value Objects: TimeSlot and UnavailabilityWindow

TimeSlot (Value Object, Chapter 4)

- date, start_time, end_time: end_time is exactly 15 minutes after start_time. A longer visit occupies consecutive slots.

An Appointment occupies the interval beginning at its scheduled_slot.start_time and running for duration_minutes. The slot's end_time marks the first 15 minutes only, so a 60-minute appointment starting at 09:00 holds 09:00 to 10:00 and blocks the three slots following its own. Rule 3 and Rule 4 test overlap against that whole interval, never against end_time.

No identity of its own; meaningful only as an Appointment's scheduled_slot. This specification computes a slot's availability at request time rather than storing it as a separate record with its own status; see Rule 3 under Consistency and validation rules.

UnavailabilityWindow (Value Object)

- doctor_id: reference to Doctor.
- clinic_id: optional. Absent means the doctor is unavailable for the window's duration at every clinic where they work.
- start, end: the unavailable range, as RFC 3339 timestamps, so one window covers part of a day or several whole days.
- reason: string, for staff reference only (vacation, conference, and similar cases).

No identity of its own; two windows naming the same doctor, range, and reason are the same fact recorded twice, replaced rather than edited when a doctor's plans change.

A doctor's true availability at a given clinic is the clinic's business hours, minus any UnavailabilityWindow overlapping the slot, minus any date and time already held by a Scheduled Appointment for that doctor.

The Booking Protocol: Requesting an Appointment

A protocol message is a defined request and the defined responses it can produce. The first one collapses the display-and-select steps of Chapter 3's Schedule Appointment use case into a single message every conforming system implements identically: the caller supplies a date range and the system selects the slot, so the choice a patient makes in the interface never becomes a protocol state the server has to hold.

RequestScheduleAppointment

Initiator: Patient, or Staff acting on a patient's behalf.

Parameters: patient_id, doctor_id, clinic_id, reason_for_visit, duration_minutes, preferred_date_range_start, preferred_date_range_end.

Preconditions: the referenced Patient and Doctor exist; the Doctor works at the referenced Clinic; preferred_date_range_start is no more than 90 days from today (Rule 1). A preferred_date_range_end past the horizon is clamped to it rather than rejected, so a patient asking for any time in the next six months gets offered the bookable part of that range.

RESPONSE: SUCCESS

AppointmentScheduled, carrying appointment_id, scheduled_slot, confirmation_number. A confirmation notification is dispatched within 60 seconds (see Notifications).

RESPONSE: ALTERNATIVE

NoAvailableSlots, carrying doctor_id, clinic_id, requested_date_range, and suggested_alternatives: other slots with the same doctor at a later date, or other doctors of the same specialty with availability inside the requested range.

RESPONSE: ERROR

ValidationError, carrying error_code and error_message. Returned when a precondition fails: an unknown patient or doctor, a doctor who does not work at the named clinic, or a date range that starts past the 90-day horizon.

The Booking Protocol: Races and Cancellation

Concurrent Reservation

Problem: two RequestScheduleAppointment messages resolve to the same open slot before either booking completes.

1. The system re-checks availability for the exact doctor, clinic, date, and time immediately before creating the Appointment record, inside the same transaction.
2. If the slot is already held by a Scheduled appointment, the system returns NoAvailableSlots to the second request, with alternatives computed at that moment.
3. No two Scheduled appointments result for the same slot. Whether an implementation prevents the second by locking, by resolving at commit, or by holding the slot briefly is left open (Chapter 10).

CancelAppointment

Initiator: the Patient who holds the appointment, or Staff on their behalf.

Parameters: appointment_id.

Preconditions: the Appointment exists and its status is Scheduled; its scheduled_slot start is at least 24 hours away.

RESPONSE: SUCCESS

AppointmentCancelled, carrying appointment_id and cancellation_timestamp. A cancellation notification is dispatched within 60 seconds.

RESPONSE: ERROR

TooLateToCancel, carrying appointment_id, hours_until_appointment, and a message stating the 24-hour notice requirement (Rule 7).

ValidationError, carrying error_code and error_message. Returned when the referenced Appointment does not exist, or when its status is anything other than Scheduled.

Consistency and Validation Rules, Part One

Every implementation must enforce these regardless of storage technology. A database schema that violates one of these rules has a defect; the specification itself stays unambiguous.

RULE 1: THE 90-DAY BOOKING HORIZON

An appointment's `scheduled_slot` date cannot be more than 90 days from the request date. Doctor schedules at Riverside are only confirmed that far out; the number is the same one the requirements workshop settled in Chapter 3.

RULE 2: BUSINESS HOURS AND SLOT ALIGNMENT

A `scheduled_slot` must fall inside the referenced clinic's `business_hours`, and so must the whole interval the Appointment occupies. Its `start_time` must land on a 15-minute boundary, and the Appointment's `duration_minutes` must be 15, 30, 45, or 60.

RULE 3: DOCTOR CAPACITY, NO DOUBLE-BOOKING

No two Scheduled appointments for the same doctor may overlap in time, regardless of which clinic each one is at. A doctor who works two clinics still has one calendar. The many-to-many relationship in the domain model describes where a doctor can work; Rule 3 governs how many places they can be at once, and the answer is always one.

RULE 4: PATIENT CALENDAR, NO OVERLAPPING APPOINTMENTS

A patient cannot hold two Scheduled appointments with overlapping times.

Consistency and Validation Rules, Part Two

RULE 5: IMMUTABLE HISTORY

Once an Appointment reaches Completed, Cancelled, or NoShow, none of its attributes change again. The record has become history.

RULE 6: TEMPORAL CONSISTENCY

`scheduled_slot` must be at or after `created_timestamp`. An appointment cannot be booked into the past.

RULE 7: CANCELLATION NOTICE

A patient must cancel at least 24 hours before `scheduled_slot` starts. The notice period exists so the clinic has time to offer the slot to another patient.

RULE 8: WHO MAY READ THE REASON FOR A VISIT

`reason_for_visit` is visible only to the patient who holds the appointment, the named doctor, and staff at the named clinic. No other actor reads it, and no notification carries it: a notification names the appointment, never the reason for it. Chapter 2's vision exists because stating a reason where other people can hear it is what kept some patients from booking at all. A system that then repeats the reason in an email, or shows it to staff at a clinic the patient never chose, has rebuilt the barrier the feature was written to remove.

Rules 1, 6, and 7 all depend on the system's clock agreeing with the patient's clock and with every other server's clock; the implementation notes later in this appendix say what that requires.

Notification Behavior

Four events trigger a notification. Each has a defined recipient, defined contents, and a delivery timing requirement, stated below against whichever moment that particular notification is timed from.

EVENT	RECIPIENT	CONTENTS	TIMING
Appointment scheduled	Patient, email	Doctor, date, time, clinic, confirmation number, cancellation instructions, the 24-hour cancellation notice	Within 60 seconds
Appointment cancelled	Patient, email	Doctor, date, time, clinic, cancellation timestamp, confirmation the slot is open again	Within 60 seconds
Appointment reminder	Patient, email	Doctor, date, time, clinic, how to cancel and rebook	24 hours before the slot, inside the hour before that mark
Appointment completed	Patient, email	Next-step instructions	Optional, by clinic policy

No row above carries `reason_for_visit`. Rule 8 keeps it out of every notification, the optional completion message included, because an email sits where anyone with the patient's screen or inbox can read it.

Notification delivery must be at-least-once: sending the same notification twice is acceptable, since a patient who reads two identical confirmation emails loses nothing. Failing to send one is not acceptable, because a patient who isn't told their appointment was cancelled will show up to a slot that no longer exists.

Enumerations

An enumeration limits an attribute to a fixed, named set of values (Chapter 4). Three attributes in this specification are enumerations, and every conforming implementation must reject a value outside the set.

Appointment.status

- `Scheduled: confirmed, waiting to occur.`
- `Completed: the visit occurred.`
- `Cancelled: the patient or staff cancelled it before it occurred.`
- `NoShow: the patient did not arrive and did not cancel.`

Doctor.specialty

`Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry (Chapter 4).`

Appointment.created_by_actor

`Patient, Staff.`

Chapter 4 warned that an enumeration frozen after the first draft eventually stops covering what the business actually does. Telehealth, when Riverside adds it, will need a fifth status value or a separate visit-type attribute; this specification does not add one speculatively before the business asks for it.

Implementation Notes

Guidance for whoever builds this, without mandating how they build it.

PERSISTENCE

Any store capable of enforcing Rules 1 through 8 is acceptable: a relational database, a document store, an event store. The rules constrain behavior; schema shape is left to whoever implements them.

ATOMICITY

Creating an Appointment must be one atomic operation: the record is fully written, or nothing is written. The requirement rules out a design that writes the Appointment and updates availability as two separate steps that could fail between one and the other.

IDEMPOTENCY

RequestScheduleAppointment is not idempotent by default; a client that resubmits an identical request after a timeout, without knowing whether the first one succeeded, can end up creating two appointments unless it supplies its own request identifier for the server to deduplicate against. Notification delivery is the opposite case: at-least-once, because duplicates there are harmless.

CLOCK SYNCHRONIZATION

Rules 1, 6, and 7 all compare timestamps across a request's origin and the server evaluating it. A deployment running more than one server must keep every server's clock synchronized, typically through the Network Time Protocol (NTP), or a patient whose local clock reads three minutes fast can appear to cancel with 23 hours and 57 minutes' notice instead of the 24 hours the rule requires. All timestamps in this specification follow RFC 3339, which requires an explicit UTC offset, as in `2026-07-25T14:30:00-04:00`, so every server parses the same string the same way regardless of locale.

TESTING BASELINE

- A valid RequestScheduleAppointment succeeds and produces exactly one Appointment inside the clinic's business hours.
- A request whose date range starts past the 90-day horizon returns ValidationError; one that only ends past it is clamped to the horizon and still books.
- A 60-minute booking holds all four consecutive slots: a second appointment starting 15, 30, or 45 minutes into it is refused under Rule 3.
- Two simultaneous requests for the same slot: one succeeds, one receives NoAvailableSlots, and no second Appointment is created.
- Cancellation inside and outside the 24-hour window produces the correct response in each case.
- A server restart mid-transaction leaves the datastore consistent with Rule 3 and Rule 4.

Versioning: Changes Proposed for v1.1

This specification is versioned the way the rest of the book argues a specification should be: changes are proposed, reviewed, and stated as a difference against the version implementations already run. None of this has been built. It is what the next version would say if Riverside's clinic operations team asked for it.

ADDITION: RECURRING APPOINTMENTS

A new entity, `RecurringSeries`: `series_id`, `patient_id`, `doctor_id`, `clinic_id`, `recurrence_rule` (for example, weekly for six occurrences), `series_status` (`Active`, `Cancelled`). Appointment gains an optional `series_id` reference. Creating a series generates individual Appointment records, but only for occurrences that fall inside Rule 1's 90-day horizon; later occurrences generate as the horizon rolls forward. Cancelling one occurrence cancels that Appointment alone, under Rule 7, without touching the series; cancelling the series cancels every future Scheduled occurrence at once.

ADDITION: MULTI-CLINIC BOOKING

In v1.0, RequestScheduleAppointment requires `clinic_id`; the patient already knows which location they want. Proposed v1.1 makes `clinic_id` optional. When it's absent, the system searches every clinic the named doctor works at and returns the earliest available slot across all of them. AppointmentScheduled's response then includes the `clinic_id` it selected, so the patient learns where to go from the response itself rather than from a choice they made earlier.

Both additions are backward compatible: every v1.0 field keeps its meaning, and a v1.0 request with `clinic_id` supplied behaves exactly as it does today. An implementation states which version of this specification it supports. A v1.0 client and a v1.1 client can therefore both be live at once during a migration, each conforming to the version it declares.

What This Specification Replaces

This appendix doesn't depend on Java, Spring Boot, or any of the other technologies that Chapter 7's skills name. A team could implement it in Go next year and a different team could reimplement it again after that, and a patient booking an appointment would notice no difference, because the protocol stayed the same while the code underneath it changed twice. That is the argument Chapter 11 makes about specifications generally, applied here to one feature in full.

CITED STANDARDS

- RFC 5322, Internet Message Format: the address syntax Patient.email must conform to.
- ITU-T E.164, the international public telecommunication numbering plan: the format Patient.phone must conform to.
- RFC 3339, Date and Time on the Internet: Timestamps: the format every timestamp in this specification follows.
- ISO 8601: the broader international date-time standard RFC 3339 profiles for internet use.

Appendix B reads RFC 791, RFC 793, and RFC 7230 as the precedent this format borrows from. This appendix is the same discipline turned inward, on a business protocol instead of a network one.